Governance Repository — Complete Analysis

Executive Summary

The **Governance** repository is a **centralized Scrumban orchestration system** that acts as a single source of truth for managing multiple AI-assisted software projects. It's innovative because it treats AI agents (Claude, Gemini, ChatGPT) as **first-class team members** in a formal development process, with defined workflows, quality gates, and cross-agent communication infrastructure.

Core Innovation: This isn't just documentation — it's an executable governance framework that synchronizes priorities, methodology, and learnings across multiple repositories and AI clients in real-time.

---

1. What It Does

Primary Functions

A. Central Priority Authority

- Maintains a cross-project backlog with BV (Business Value) scoring
- Enforces a three-stage pipeline: Ideation → Refining → Ready
- Distributes prioritized work to multiple project repositories

B. Methodology Enforcement

- Defines the complete Scrumban execution loop (outer/inner/cleanup loops)
- Implements quality gates: DOR (Definition of Ready) and DOD (Definition of Done)
- Establishes autopilot mode for AI-driven execution

C. Knowledge Management

- Captures anti-patterns and lessons learned across all projects
- Tracks retrospectives (triggered every 10 completed stories)
- Maintains decision logs and architectural knowledge

D. Multi-Repository Synchronization

- Auto-syncs governance files to all connected project repos
- Updates Excel dashboards via COM automation
- Pushes structured data to Google Sheets via Tether
- Uses git post-commit hooks for automatic propagation

E. Cross-Agent Communication Hub

- Integrates with Tether (messaging infrastructure) to enable AI-to-AI communication
- Enables different AI clients (Claude, Gemini, Grok, ChatGPT) to work on the same project
- Plans evolution to "Governance-as-Agent" — an active PO that sends tasks via Tether

---

2. How It's Innovative

A. AI-First Development Process

Traditional Approach:

- Developer uses AI as a glorified autocomplete
- No formal handoff between AI sessions
- Context loss between conversations

Governance Repo Approach:

- AI agents are formal team members with defined roles
- Formal pipeline: INBOX → BACKLOG → TODO-Today → DONE-Today
- Persistent memory files ensure context preservation
- Quality gates prevent AI from shipping broken code

B. The Three-Loop System

1. Outer Loop (Human-Driven)

```
INBOX.md → /triage → BACKLOG.md → /workflow → TODO-Today.md
```

- Humans provide strategic direction
- Planning rounds populate execution queues
- Always ends with mandatory quality checks

2. Inner Loop (AI Autopilot)

```
While semaphore == "run":
  - Read next unchecked task
  - Verify DOR gate
  - Execute task
  - Run cleanup sub-loop
  - Commit if DOD passes
  - Move to DONE-Today.md
```

- Dumb executor model (no AI discretion)
- Semaphore-controlled (human can pause anytime)
- Checks `.autopilot` file before EVERY task

3. Cleanup Sub-Loop (Nested Quality Gate)

```
After every implementation:
 - Run greenlight quality check
 - Fix all Low findings
 - If Medium/High findings exist:
   → Pause autopilot
   → Notify human
   → STOP
```

- No commits without clean quality audit
- Prevents "AI slop" from entering codebase

C. Real-Time Cross-Agent Synchronization

The Tether Integration:

```
Governance Repo
   │
   ├──> sync-repos.ps1 (git post-commit hook)
   │
   ├──> Git Repos (SVG-PAINT, Tether, etc.)
   ├──> Excel (COM automation)
   └──> Google Sheets (via Tether HTTP API)
        │
        └──> Gemini, ChatGPT, Grok (read/write backlog)
```

This enables:

- Claude working in VSCode while Gemini reads priorities from Sheets
- Retro insights flowing back from external AI agents
- Priority changes propagating to all active agents instantly

D. Quality Gates as Hard Constraints

DOR (Definition of Ready) — Before Implementation:

- User Stories with Acceptance Criteria
- Spec document with `/sc:spec-panel` score ≥ 7.0
- Architecture decisions documented

- Dependencies identified
- Test strategy defined
- Size estimated (S/M/L)

Bug DOR-lite: Lightweight for hotfixes (root cause + fix plan + regression test)

DOD (Definition of Done) — Before Deployment:

- All tests pass
- Quality audit clean (only Low findings allowed)
- Service-first architecture
- Clean git commit
- Application starts
- No regressions

Enforcement: The queue tail ALWAYS includes DOD checks. AI cannot bypass.

---

3. Key Files and Their Roles

Core Methodology Files

File	Purpose	Used By
CLAUDE-LOOP.md	Complete loop definition (canonical)	All projects include by reference
DOR.md	Gate before implementation	/sc:workflow checks this
DOD.md	Gate before deployment	Queue tail enforces
KNOWN_PATTERNS.md	Anti-patterns from all projects	AI consults before writing code

Planning & Execution Files

File	Purpose	Updated By
BACKLOG.md	Cross-project backlog (Ideation/Refining/Ready)	Human + /sc:brainstorm + /sc:spec-panel
ROADMAP.md	Strategic initiatives with BV scoring	Human (PO)
TODO-Today.md	Execution queue (per project)	/sc:workflow generates, autopilot consumes
DONE-Today.md	Completion log	Autopilot moves items here
INBOX.md	Raw input dump	User writes here first

Infrastructure Files

File	Purpose	Trigger
sync-repos.ps1	Syncs governance to repos + Excel + Sheets	git post-commit hook
.autopilot	Semaphore file (run/pause/stop)	Read before EVERY task
.claude-action	Human notification channel	AI writes findings/blockers here
MEMORY.md	Session-to-session context	Updated at session end
KILO_BOOTSTRAP.md	Agent onboarding guide	Used when adding new AI agents

Documentation & Learning

File	Purpose	Frequency
Gemini.md	Complete cross-agent integration guide	Living document
retros/	Retrospective history	Every 10 completed stories
requirements/	Spec documents	Per feature/epic
templates/	Reusable CLAUDE.md snippets	Used to bootstrap new projects

---

4. Functions and Mechanisms

A. Scrumban Commands (`/sc:` prefix)

Planning Commands:

```bash
/sc:brainstorm "[idea]" --depth deep
# Output: requirements/SPEC.md (moves Ideation → Refining)

/sc:spec-panel requirements/SPEC.md --mode critique --focus requirements
# Gate: Score ≥ 7.0 (moves Refining → Ready)

/sc:workflow requirements/SPEC.md --strategy systematic --depth deep
# Output: Populates TODO-Today.md with queue items
```

Execution Commands:

```bash
/sc:analyze "<files>" --focus quality
# Runs quality audit, returns findings (L/M/H severity)

/sc:cleanup --type all
# Fixes Low findings automatically

/sc:test --coverage
# Runs test suite with coverage report

/sc:git --smart-commit
# Atomic commit with descriptive message
```

B. Triage System (`/triage`)

Input: Anything in INBOX.md **Output:** Classified and routed

```
Bug/Hotfix    → TODO-Today.md (fast-track)
Feature       → BACKLOG.md#Ideation
Enhancement   → BACKLOG.md#Ideation
Question      → Answered conversationally (no queue item)
```

Critical Invariant: Queue-first. Even obvious bugs get a queue item BEFORE implementation.

C. Graduation Pipeline

```
INBOX.md
  ↓ /triage
BACKLOG.md#Ideation
  ↓ /sc:brainstorm (generates requirements)
BACKLOG.md#Refining
  ↓ /sc:spec-panel (validates spec, score ≥ 7.0)
BACKLOG.md#Ready
  ↓ /sc:workflow (generates execution queue)
TODO-Today.md
  ↓ autopilot (executes, commits, deploys)
DONE-Today.md
```

D. Semaphore Control

Three states:

- `run` — autopilot executes
- `pause` — autopilot stops at current item, waits
- `stop` — autopilot terminates completely

Check frequency: Before EVERY queue item (not just session start)

Who writes it:

- Human (manual override)
- AI (when Medium/High findings detected)

E. Quality Enforcement Mechanism

Cleanup Sub-Loop (nested inside Inner Loop):

```python
# Pseudo-code of the enforcement mechanism
def execute_task(task):
    implement(task)

    while True:
        findings = run_greenlight()

        low_findings = [f for f in findings if f.severity == "Low"]
        medium_high = [f for f in findings if f.severity in ["Medium", "High"]]

        if not low_findings and not medium_high:
            break  # Clean → proceed to commit

        if low_findings:
            fix_all(low_findings)
            continue  # Re-check

        if medium_high:
            write_to_claude_action(medium_high)
            write_to_autopilot("pause")
            STOP  # Human review required

    commit_atomic()
    move_to_done()
```

Key principle: AI NEVER downgrades findings. Medium/High always pauses.

F. Retro Trigger System

Counter: Tracked in project's MEMORY.md **Frequency:** Every 10 completed stories **Scope:** Examines:

- Workflow friction
- Bugfixing quality
- Memory file usefulness

- Loop robustness
- Tooling gaps
- Quality gate effectiveness

Output: Actionable items → BACKLOG.md#Ideation or CLAUDE.md rule updates

G. Cross-Project Sync (sync-repos.ps1)

Trigger: git post-commit hook **Three Destinations:**

1. Git Repos (file copy):

```powershell
$Files = @(
    "CLAUDE-LOOP.md",
    "DOR.md",
    "DOD.md",
    "KNOWN_PATTERNS.md",
    "BACKLOG.md",
    "ROADMAP.md"
)

foreach ($repo in $ConnectedRepos) {
    Copy-Item $Files "$repo/governance/"
}
```

2. Excel (COM automation):

```powershell
$Excel = New-Object -ComObject Excel.Application
$Workbook = $Excel.Workbooks.Open("exports/Governance-Dashboard.xlsx")
# Parse BACKLOG.md → write to "Backlog Items" sheet
# Parse ROADMAP.md → write to "Roadmap" sheet
# Parse git log → write to "Sync Log" sheet
$Workbook.Save()
```

3. Google Sheets (via Tether HTTP API):

```powershell
```

```
$Payload = @{
    table = "governance_backlog"
    data = $ParsedBacklogItems
} | ConvertTo-Json

Invoke-RestMethod -Uri "http://localhost:7890/collapse" -Method POST -Body $Payload
# Sheets AppScript polls Tether, pulls data, writes to spreadsheet
```

---

5. Governed Projects

Project	Path	Scope	Status
SVG-PAINT	D:\Temp\git\SVG-PAINT	Etsy SVG asset generation platform	Architecture redesign (Refining)
Tether	D:\Temp\git\Tether	LLM-to-LLM messaging bridge	Operational (v1.0)
Future	TBD	Any new initiative	Bootstrap from templates/

How projects connect:

Every governed project's CLAUDE.md includes:

```markdown
## Governance

This project is governed by `D:\Temp\git\Governance`.
- Loop definition: `Governance/CLAUDE-LOOP.md`
- DOR gate: `Governance/DOR.md`
- DOD gate: `Governance/DOD.md`
- Known patterns: `Governance/KNOWN_PATTERNS.md`
- Cross-project backlog: `Governance/BACKLOG.md`
```

---

6. Future Vision: Governance-as-Agent

Current State: Passive files that humans and AI read

Planned Evolution:

```
Governance Repo → Active PO Agent
    │
    ├─> Reads BACKLOG.md priority order
    ├─> Sends prioritized tasks via Tether threads
    │   - Thread: "svg-paint"   → Claude
    │   - Thread: "tether-ui"   → Gemini
    │   - Thread: "data-pipeline" → ChatGPT
    │
    ├─> Receives retro insights via Tether messages
    ├─> Updates KNOWN_PATTERNS.md automatically
    └─> Adjusts priorities based on cross-project feedback
```

Key Components Needed:

- Tether thread-per-project
- Governance CLI that reads BACKLOG.md and posts to Tether
- MCP server for governance operations
- Auto-commit scheduler for daily snapshots

---

7. Anti-Patterns Captured

From **KNOWN_PATTERNS.md** (cross-project lessons):

Category	Anti-Pattern	Impact	Origin
Config	Evaluating settings at module import	Values freeze, ignore runtime changes	SVG-PAINT
Error Handling	Catching bare `Exception`	Hides system exits, OOM	SVG-PAINT
Data	`dict.get(key, default)` when value can be None	Returns None not default	Tether
Process	Semaphore read once at session start	Misses mid-session pause signals	SVG-PAINT
Frontend	Hardcoded hex colors	Violates theme consistency	SVG-PAINT
Infra	Bare `python` in scripts	Wrong venv resolution	Tether

Evolution: Project-specific patterns stay in project repos. Patterns that recur across ≥2 projects graduate to Governance/KNOWN_PATTERNS.md.

---

8. Technical Architecture

Data Flow

```
┌─────────────────────────────────────────────────────────────┐
│                    Human (PO Role)                          │
└─────────────────────────────────────────────────────────────┘
                              │
                              ▼
┌─────────────────────────────────────────────────────────────┐
│                    INBOX.md (raw input)                     │
└─────────────────────────────────────────────────────────────┘
                              │
                              ▼ /triage
┌─────────────────────────────────────────────────────────────┐
│                    BACKLOG.md (3-stage funnel)              │
│  ┌──────────────┬──────────────┬──────────────┐            │
│  │ Ideation     │ Refining     │ Ready        │            │
│  └──────────────┴──────────────┴──────────────┘            │
└─────────────────────────────────────────────────────────────┘
                              │
                              ▼ /sc:workflow
┌─────────────────────────────────────────────────────────────┐
│                    TODO-Today.md (execution queue)          │
│                    - [ ] Task 1                             │
│                    - [ ] Task 2                             │
│                    - [ ] Quality tail (DOD enforcement)     │
└─────────────────────────────────────────────────────────────┘
                              │
                              ▼ autopilot (checks .autopilot semaphore)
┌─────────────────────────────────────────────────────────────┐
│                    AI Executor                              │
│  1. Read task                                               │
│  2. Verify DOR                                              │
│  3. Implement                                               │
│  4. Run cleanup sub-loop (quality gate)                     │
│  5. Commit (if DOD passes)                                  │
│  6. Move to DONE-Today.md                                   │
└─────────────────────────────────────────────────────────────┘
                              │
                              ▼
```

```
│      DONE-Today.md (completion log)                │
│      - [x] Task 1 (2026-03-01 14:23)               │
│      - [x] Task 2 (2026-03-01 15:47)               │
└──────────────────────────────────────────────────┘
                  │
                  ▼ Every 10 completed stories
┌──────────────────────────────────────────────────┐
│      retros/ (retrospective analysis)              │
└──────────────────────────────────────────────────┘
```

Sync Architecture

```
Governance Repo (D:\Temp\git\Governance)
    │
    │ (git post-commit hook)
    ├──> sync-repos.ps1
    │
    ├──────> Connected Repos
    │        ├──> D:\Temp\git\SVG-PAINT\governance\
    │        └──> D:\Temp\git\Tether\governance\
    │
    ├──────> Excel Dashboard
    │        └──> exports/Governance-Dashboard.xlsx
    │             ├── Backlog Items (sheet)
    │             ├── Roadmap (sheet)
    │             └── Sync Log (sheet)
    │
    └──────> Google Sheets (via Tether)
             └──> localhost:7890/collapse (HTTP POST)
                  └──> tether.db (SQLite)
                       └──> Apps Script (polls Tether)
                            └──> Google Sheets
                                 ├── Governance Backlog (tab)
                                 └── Roadmap (tab)
```

---

9. Why This Is Innovative

A. Treats AI as a Formal Team Member

- Not a tool, but an **autonomous executor** with defined responsibilities
- Bounded autonomy: can execute, cannot prioritize
- Formal handoffs via pipeline stages
- Quality gates prevent "garbage in, garbage out"

B. Solves the "AI Context Loss" Problem

- MEMORY.md preserves state between sessions
- DONE-Today.md creates execution history
- KNOWN_PATTERNS.md captures institutional knowledge
- Retros extract learnings systematically

C. Multi-Agent Coordination

- Multiple AI clients (Claude, Gemini, Grok) work on same backlog
- Tether provides cross-agent messaging
- Google Sheets enables non-MCP clients to participate
- Planned: Governance-as-Agent for active coordination

D. Quality-First Execution Model

- No commits without greenlight passing
- Medium/High findings ALWAYS pause autopilot
- Queue tail is mandatory (cannot skip DOD)
- Human-in-the-loop for any quality degradation

E. Single Source of Truth

- One backlog for all projects
- One roadmap with BV scoring
- One set of lessons learned
- Auto-sync ensures consistency

F. Executable Documentation

- Not just markdown files you read
- Files that control execution (semaphores, queues)
- Scripts that propagate changes (sync-repos.ps1)
- Hooks that enforce gates (post-commit)

---

10. Comparison to Traditional Approaches

Aspect	Traditional Agile	Traditional AI Usage	Governance Repo
Backlog	Jira/Linear	Chat history	BACKLOG.md with 3 stages
Sprint Planning	Meeting	Ask AI each time	/sc:workflow generates queue
Quality Gates	Code review	Manual testing	Automated DOR/DOD + greenlight
Context Preservation	Team memory	Copy-paste prompts	MEMORY.md + DONE-Today.md
Cross-Agent Comm	Slack/Teams	N/A	Tether messaging
Lessons Learned	Wiki	Forgotten	KNOWN_PATTERNS.md
Definition of Done	Checklist	Honor system	Enforced queue tail
Priority Authority	PO discretion	Implicit in prompts	Explicit in BACKLOG.md order

---

11. Key Innovations Summary

1. **Three-Loop System** — Outer (human planning) + Inner (AI execution) + Cleanup (quality enforcement)
2. **Semaphore-Controlled Autopilot** — Human can pause AI at any point
3. **Mandatory Quality Tail** — DOD checks are last queue items, cannot be skipped
4. **Cross-Project Knowledge Base** — KNOWN_PATTERNS graduates patterns from projects
5. **Real-Time Multi-Repo Sync** — git hook → PS script → 3 destinations (repos/Excel/Sheets)
6. **Multi-Agent Backlog** — Claude, Gemini, Grok all read from same priority list
7. **Executable Governance** — Files control execution, not just document it
8. **Retro-Driven Improvement** — Every 10 stories triggers systematic learning
9. **DOR/DOD as Hard Gates** — AI cannot bypass, enforced by workflow commands
10. **Tether Integration** — Messaging bus for AI-to-AI communication

---

12. Files That Make It Work

Must-Read for Understanding

1. **CLAUDE-LOOP.md** — The complete execution model
2. **BACKLOG.md** — See the 3-stage funnel in action
3. **DOR.md + DOD.md** — The quality gates

4. **sync-repos.ps1** — How governance propagates
5. **Gemini.md** — Cross-agent integration (detailed technical guide)

Configuration Files

- `.autopilot` — Semaphore (run/pause/stop)
- `.claude-action` — AI → Human notification channel
- `~/.claude.json` → MCP server config (connects Claude to Tether)
- `tether.db` — Canonical SQLite database (all agents share)

Learning Files

- `KNOWN_PATTERNS.md` — Anti-patterns (AI must consult)
- `retros/` — Retrospective history
- `MEMORY.md` — Session-to-session context

Conclusion

The Governance repository is a **production-grade AI orchestration system** that solves three hard problems:

1. **Context Preservation** — AI doesn't lose track across sessions
2. **Quality Control** — AI can't ship broken code
3. **Multi-Agent Coordination** — Multiple AIs work together coherently

It's innovative because it treats AI as a **bounded autonomous agent** within a formal Scrumban process, not just a chatbot or autocomplete tool. The combination of execution loops, quality gates, cross-project sync, and Tether-based messaging creates a system where AI can operate reliably at scale across multiple repositories and multiple AI clients.

The killer feature: It's not vaporware. This is a working system with real projects (SVG-PAINT, Tether) governed by it right now, with concrete plans to evolve into a Governance-as-Agent PO that actively coordinates work across AI clients via Tether messaging.